1 Lesson 16-10-26

Let's get back to dependencies, which we first introduced in 6.3.

1.0.1 Data hazards

Each time an operand involved in a dependency is accessed in a different order than the original one, there could be a hazard.

This means that the program output may become wrong.

Data hazards can be classified in three categories, based on the type of two instructions i, j executed in order:

- i writes, j reads: **RAW** (*Read After Write*), which stems from actual data dependencies;
- i writes, j writes **WAW** (*Write After Write*) which stems from an output (name) dependency;
- i reads, j writes **WAR** (*Write After Read*) which stems from an antidependence (name).

We can summerize this with a graph:

	i writes	i reads
j writes	**WAW** (output)	**WAR** (antidependency)
j reads	**RAW** (data)	//

1.1 Control dependencies

A control dependency occurs when an instruction depends on a branch.

- An instruction that is control dependent on a branch cannot be moved before the branch (so that its execution is no more controlled by the branch).
- An instruction that is not control dependent on a branch cannot be moved after the branch (so that its execution become dependent on the branch).

Normally, preserving control dependencies is a sufficient condition for preserving the program correctness. But there are cases in which the reverse is not true.

What actually has to stay fixed is:

- *Exception behavior*;
- *Data flow*;

1.1.1 Exception behavior

Exception behavior conservation ensures that any change in the order of instruction execution must not change how exceptions are raised in the program.

Take for instance:

```
add x2, x3, x4
beqz x2, L1
ld x1, 0(x2)

L1: ...
```

The `ld` instruction can raise an exception, so it cannot be moved before the branch, as an exception caused by the load can then happen no matter whether the branch is taken or not.

1.1.2 Data flow

Data flow is the actual flow of data among instructions that produce results and consume them. Data flow must be preserved.

Take for instance:

```
add x1, x3, x4
beqz x2, L1
sub x1, x5, x6

L1:
  or x7, x1 ,x8
```

In this example, the value of `x1` used by the `or` instruction must be that produced by the `add` if the branch is taken, or that produced by the `sub` if it is not taken.

There are cases in which it is possible to violate the control dependence without affecting the exception behavior or the data flow. These are cases where:

- The moved instruction cannot generate exceptions;
- The program results are not changed anyway.

By doing this, the compiler speculates, i.e., bets on the branch not to be taken.

1.2 Dynamic scheduling

We introduced the concept of *Instruction Level Parallellism* (**ILP**) (6.2), with the associated *dependencies*, and even introduced some *static* techniques (6.2.1). Now we look at the *dynamic* side of things, including *out-of-order* execution.

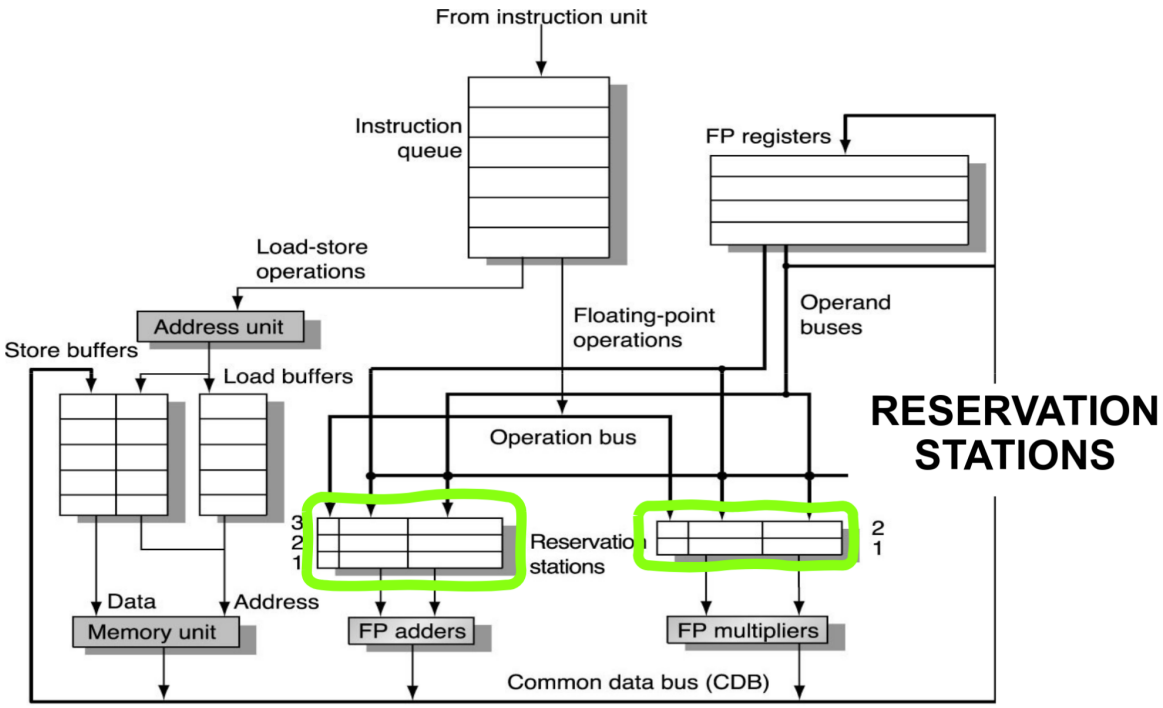
To summarize, dynamic scheduling is a technique aimed at rearranging in hardware the instruction execution order to reduce the pipeline stalls while maintaining data flow and exception behavior.

Assuming the basics of dependencies and out of order execution are covered, we move straight to *Tomasulo*'s algorithm.

1.2.1 Tomasulo's algorithm

Robert Tomasulo was the architect of the IBM 360/91 floating point unit: his ideas were first published in a paper of 1967. The same ideas were then re-used when the first superscalar processors were built. The key ideas in the Tomasulo's algorithm are to:

- Track operand availability;
- Rename registers (which we might have mentioned when talking about name dependencies).


From instruction unit
Instruction queue
FP registers
Load-store operations
Address unit
Store buffers
Load buffers
Floating-point operations
Operand buses
RESERVATION STATIONS
Operation bus
3
2
1
Reservation stations
2
1
Data
Address
Memory unit
FP adders
FP multipliers
Common data bus (CDB)